DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

LISPing Your Functions

CMSC 124 Problem Set 3

Prepared by: Rene Andre Bedonia Jocsing

Published: October 29, 2026 **Deadline:** November 11, 2026

Suppose you run this:

```
(let ((factor 2))
  (let ((f (lambda (n) (* n factor))))
    (let ((factor 10))
      (funcall f 3))))
```

The answer is 6.

Read it again. By the time `f` runs, the nearest `factor` in sight is 10, and 30 is the answer a great many people write down. Lisp says 6, because `f` didn't look up `factor` when it ran. It captured `factor` when defined. The binding it caught held 2. The `factor` bound to 10 arrived later, and `f` never saw it.

That's the closing idea of Unit 8. Activity 14 traced it through a C++ lambda. Create the function, return it from the factory, call it somewhere else, and watch it still reach the variable that made it. This problem set is where you make that behavior yours by building the machinery that produces it.

You'll do it in two halves. First you'll predict 16 Lisp forms before running any of them, four answers per form, 64 predictions in total. Then you'll write a small evaluator for a toy language in Common Lisp, in four stages, ending with the stage that decides whether the form above is 6 or 30.

This is a paired, asynchronous assignment. It runs on GNU CLISP alone, so installation can't accidentally become the assignment. Most of you meet Lisp for the first time here, so Part 1 also tells you which book to read while you work. This manual carries the rules the 16 cases grade. The book below teaches the language.

Why The Four Fields?

Common Lisp calls anything meant for evaluation a **form**. A form can be an atom, such as `3` or `factor`, or a nonempty list. A nonempty list used as code is a **compound form**. That same list can also be data. This shared representation lets the same parentheses mean "call this" or "here, have this list." The `kind` field records what evaluation returned.

Take a form that looks like arithmetic:

```
(quote (+ 1 2))
```

The value is the three-element list `(+ 1 2)`, whose first element is the symbol `+` and whose other two are numbers. `quote` is a special operator. It returns its argument untouched (like you're quoting **cs1** verbatim). Meanwhile `(+ 1 2)` on its own is 3, an integer, which takes `none` for `length`.

So each case asks four independent questions:

Field	Question
`value`	What does the form evaluate to, written in the grader's notation?
`kind`	Which of the course's kinds is the result?
`length`	How many top-level elements does it hold, or `none`?
`operator`	For a compound form, is its head a function, a special operator, or a macro? An atomic form takes `none`.

`nil` *Is Three Things at Once*

The `kind` field exists because Lisp's most common value refuses to sit in one box. `nil` is the empty list, a symbol, and false. You need all three.

```
(symbolp nil)   ; T
(listp nil)     ; T
(null nil)      ; T
```

No single word describes `nil`. Any one choice is wrong twice. So the course pins a vocabulary that splits it out:

`kind`	What it covers
`null`	`nil`, and only `nil`
`cons`	a list with at least one element
`symbol`	a symbol that isn't `nil`, including `t`
`integer`	a whole number
`ratio`	an exact fraction such as `3/4`
`string`	a string
`character`	a character
`function`	a function object

`nil` reads `null`, never `symbol` and never `cons`. `t` reads `symbol`. Watch what that does to `length`. A sequence has a length and everything else answers `none`, and `nil` is a sequence, the empty one, so its length is `0` rather than `none`. That combination, `null` with length `0`, turns up in three of the 16 cases.

The vocabulary above differs from what `type-of` returns. The Lisp standard lets each implementation answer `type-of` however it likes, so `(type-of 3)` can read `FIXNUM` on one build and `INTEGER` on another. A published answer table built on it would be wrong somewhere. Hence, the kinds above are defined in `scripts/support.lisp` and mean the same thing everywhere.

The `operator` *Field Is Where Prefix Evaluation Gets Graded*

A compound form puts its head first and its remaining subforms after it. That uniformity hides a distinction. Three different things sit in the head position across these cases. Each one follows a different rule.

A **function** evaluates every argument, in order, before it runs. `+`, `car`, `cons`, `mapcar`, and `funcall` are functions. When you write `(+ 1 (* 2 3))`, the inner multiplication finishes first and `+` receives only its value.

A **special operator** decides for itself which of its subforms to evaluate and when. `if` evaluates the condition and then one branch. `let` runs the initializers first, binds the variables, and only then evaluates the body. You can't write any of these as functions because a function's arguments are already evaluated by the time it gets them.

A **macro** rewrites the form into a different one. The new form gets evaluated instead. `defun` is a macro. So is `lambda`.

For the course's `operator` field, ask in this order. Is the head a special operator? If not, is it a macro? If not, is it a function? First yes wins.

That order settles the overlap. The standard permits an implementation to supply both a special operator and a macro for one symbol, in section 3.1.2.1.2.2, which says an implementation "is free to implement any macro operator as a special operator, but only if an equivalent definition of the macro is also provided." CLISP takes that permission for `and`, `or`, `when`, `unless`, `cond`, and `case`. All six answer yes to both tests. Asking about special operators first is what settles them, so all six read `special-operator` in this problem set. This course picked that rule. One of the 16 cases lands on it.

Learning Objectives

By the end of this problem set, you should be able to:

- Predict the value of a prefix form, including which subforms get evaluated and in what order
- Tell an atom from a list, and explain why `nil` is both a symbol and the empty list without contradicting yourself
- Classify a form's head as a function, a special operator, or a macro, and say what changes about evaluation in each case
- Write a recursive function over a list and identify its base case
- Pass a function as a value using `lambda`, `#'`, and `funcall`, and explain why Common Lisp needs the last two
- Trace a variable to its binding through a chain of environments
- Explain the difference between a special variable rebound by `let` and a lexical variable captured by a closure, using a form where the two disagree
- Implement an evaluator whose closures capture the environment they were defined in, and say which line makes them lexical

---

The Pair

I assign pairs at random across every section. You don't choose your partner. You probably won't work with your laboratory partner. If the pooled roster has an odd number of students, one trio receives the same work.

The published pairing sheet is authoritative. Both members predict all 16 cases before comparing answers. Splitting the table in half defeats the part that asks you to reason about every form. After the prediction commit you may divide the evaluator, but review each other's code before the final run because the responsibility for it belongs to both of you.

You won't always be free at the same hours. This is asynchronous work, so the repository is where you meet. Push small commits, say what you did in the message, and open an issue on the fork as soon as you're stuck. Replies can take until tomorrow. If your partner goes quiet for a couple of days, or the two of you can't find any overlapping time, tell me while there's still most of the window left. The commit

history already records who did what. That record is your case. What I can't do is fix it the night before the deadline.

This belongs to the lecture component. The laboratory booking, progress-report, and defense rules don't apply. The lecture deadline is strict.

CLISP Setup

You need GNU CLISP and Git. The syllabus specifies CLISP, which produced the published answers. Every command in this repository runs through `clisp` on the command line. All work happens on the command line. GNU CLISP and Git are the whole toolchain.

GNU CLISP's last release was 2.49, in July 2010, but the three platforms below ship builds that are years apart. The Tested Toolchains section of the starter's `README.md` explains why the gap is harmless.

Windows

Install Git first, from PowerShell:

```
winget install --id Git.Git --exact
```

Git for Windows brings **Git Bash**, a terminal that understands the Unix commands this manual uses. Open it from the Start menu. Run every later command in this manual in Git Bash. The PowerShell lines in this section are the only ones that belong in a PowerShell window. PowerShell has no `chmod` for `./check.sh`, so a command pasted into the wrong window fails in a way that looks like a broken starter.

There's no winget package for CLISP, so download the official archive and unpack it. Still in PowerShell:

```
$url = "https://downloads.sourceforge.net/project/clisp/clisp/2.49/" +
       "clisp-2.49-win32-mingw-big.zip"
curl.exe -sSL -o "$env:TEMP\clisp.zip" $url
Expand-Archive -Path "$env:TEMP\clisp.zip" -DestinationPath "C:\clisp" -Force
[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path',
  ↪  'User') + ';C:\clisp\clisp-2.49', 'User')
```

Use `curl.exe` and not PowerShell's `Invoke-WebRequest`. That host answers `Invoke-WebRequest` with a 145 KB web page. `Expand-Archive` then fails on that page. The `.exe` matters too, because plain `curl` in PowerShell is an alias for `Invoke-WebRequest`.

`Expand-Archive` creates `C:\clisp\clisp-2.49` while the last line adds `C:\clisp\clisp-2.49` to your `PATH`. Close Git Bash and open it again so it picks up the new `PATH`.

Ubuntu

The Ubuntu repository carries CLISP:

```
sudo apt update
sudo apt install -y clisp
```

macOS

Install CLISP with Homebrew:

```
brew install clisp
```

Verify the Runtime

In Git Bash, or your terminal on Ubuntu and macOS, run:

```
clisp --version
```

The first line reports a CLISP version. Any of the three builds above works. If the command isn't found, fix the installation before you clone the starter.

Run one more check now. By Day 5 you need the runtime working:

```
echo '(format t "hello~%")' > hello.lisp
clisp -q -norc hello.lisp
```

On Ubuntu and macOS that prints `hello`. On Windows it prints `*** - Win32 error 267 (ERROR_DIRECTORY): The directory name is invalid.` That error comes from the 2010 build, which can't load a Lisp file by path in any directory. That limitation also breaks running a script as an argument. The starter laboriously avoids both, and the work has already been done for you, so this assignment is unaffected. The Repository section explains this Windows gotcha.

Get the Starter

Fork cmsc-124-ps3-starter and clone your fork. A fork is a snapshot. A fix I push to the starter mid-window never reaches it, so what you were graded against can't change under you.

```
git clone https://github.com/<your-account>/cmsc-124-ps3-starter.git
cd cmsc-124-ps3-starter
chmod +x run check.sh lint
./check.sh
```

One of you forks the repository under your account, keeps the fork public, and adds the other member as a collaborator, since the fork is the pair's deliverable. Then open the Actions tab on the fork and enable workflows. GitHub disables them on every fork, so that tab shows a short notice and a button. GitHub documents the rule and the fix under Workflows in forked repositories. The workflow run is a deliverable, so don't discover this on Day 11.

A fresh starter reports `1/91 checks passed` and exits 1. That red run is the correct starting state. Sixty-four checks belong to the prediction table, twenty-five to `src/evaluator.lisp`, and one each to `REASONING.md` and `ANALYSIS.md`. The analysis check stays failed until the file lands inside the 300 to 450 word range with the placeholder gone.

The one starting pass is `evaluator_loads`, which asks only whether `src/evaluator.lisp` reads without a syntax fault. The stubs manage that on the first commit. A stub that signals an error when called is still valid Lisp. Read `1/91` as the correct starting state. Nothing has been done yet.

The Repository

```
cases/cases.lisp          the 16 forms, stored unevaluated
predictions.tsv           your four predictions for every form
```

```
REASONING.md                the rule behind every case, written before you run
src/evaluator.lisp          the eight functions you implement
tests/expected.tsv          every published expected result
tests/check-all.lisp        the complete public automated checker
scripts/support.lisp        the notation, the kinds, and the head classification
scripts/run-case.lisp       the one-case runner
scripts/validate-predictions.lisp  the prediction table's form checks
scripts/lint-predictions.lisp      what ./lint runs
run  lint  check.sh         the course run contract
ANALYSIS.md                 the final explanation you write together
.github/workflows/test.yml  your CI run of the same checks
```

You edit four files: `predictions.tsv`, `REASONING.md`, `src/evaluator.lisp`, and `ANALYSIS.md`. You also replace the starter's `README.md` with your own. Keep the case corpus, expected table, runner, grader, and the `scripts/` helpers unchanged. The grader evaluates all 16 forms before scoring and stops if CLISP disagrees with the published expected table.

`check.sh` is the complete public automated check for the predictions, the evaluator, and the analysis word range. The same script runs on your machine, in your GitHub Actions workflow, and during grading. There's no hidden test suite. The rubric also judges the analysis itself, the commit history, and the submitted workflow runs. A `91/91` result satisfies the local test requirement. Your final grade still depends on those rubric rows.

Your repository being a fork makes the fixture check cheap. Grading starts from the starter tag and reads the fork's diff against it, so an edited fixture is visible as a diff in the open.

That policy creates an awkward fact for a prediction assignment. The public file `tests/expected.tsv` contains the answers, because grading can't depend on secret results. Don't open it until your prediction commit exists. Git records when your table changed. It can't prove which files you read first, so this part depends on your honesty.

Why the Scripts Say `ps3-load`

Open `check.sh`. It starts CLISP with a small bootstrap that defines a function called `ps3-load`. Every Lisp file here uses that in place of the standard `load`.

That's the Windows problem from the setup section, solved. Since `load` fails on the 2010 Windows build for every path it's given, `ps3-load` opens the file first and hands `load` the already-open stream, which works everywhere. A single contract covers every platform. You never call it yourself. Now the line won't look strange when you read it.

What design pattern is this applying?

Part 1: Predict Before Execution

Open `cases/cases.lisp` and `predictions.tsv` side by side. The prediction file starts like this:

```
id  value   kind    length  operator
P01 TODO    TODO    TODO    TODO
```

The case file looks like this:

```
(cons "P01" '(+ 1 (* 2 3)))
```

Note two pieces of that line before you read fifteen more. The leading quote `'` hands back the form itself, which is how the corpus stores all 16 cases in one list and evaluates them one at a time. A form your program can hold, pass around, and evaluate later is an ordinary value. That idea is most of why Lisp looks the way it does. The `cons` pairs the case id with the form.

Read the rules below and the notation after them, fill the table, then commit it. Only then run anything.

Say Why, Before You Run

The table records sixteen answers. The reasoning behind them is what the grade weighs. It lives in `REASONING.md`. The starter ships that file with one heading per case and a placeholder under each. Replace every placeholder with one to three sentences that state the rule behind that case's four fields.

Write the reasoning in the same commit as the table, before you run a case or open `tests/expected.tsv`. A pair that fills the table from the public answer file but can't say why each row reads the way it does has done no work. The history shows when each line appeared.

Here is an entry for a case outside the corpus. Say the corpus held `(car '(9 8))`:

```
## car-top

`(car '(9 8))` heads with the function `car`. The quote hands `car` the list
unevaluated, and `car` returns its first element, 9. That is an integer, so the
length is `none` and the head is a function.
```

That entry states the rule. Contrast it with a restated answer, which the rubric scores lower: "The value is 9, the kind is integer, the length is none, and the operator is function."

After you run, add a short correction under each case you predicted wrongly. Say what you predicted, what CLISP printed, and which rule you misread:

```
## car-top

I predicted the value `(9,8)` and CLISP printed `9`. I read `car` as returning
the list that starts with the first element, when it returns the element alone.
The kind, length, and operator I got right.
```

Those correction entries belong in the correction commits. Write reasoning from the rule in the manual and the reading, never from a run you haven't done yet.

The Rules You Need

An **atom** is any value that isn't a cons cell. Numbers, symbols, strings, and `nil` are the atoms in these cases. A **cons cell** holds two references. A list is a chain of cons cells. `(cons 1 '(2 3))` makes a three-element list by putting 1 in front of a two-element one.

Evaluation here has three cases. A number evaluates to itself, while a symbol evaluates to its binding. A compound form follows the rule selected by its head. That's what the `operator` field asks about.

`quote` returns its argument untouched. The reader turns a leading `'` into a `quote` form before evaluation ever starts, so `'(a b c)` and `(quote (a b c))` are the same form and both have `quote` as their head. That matters for the `operator` field on one case.

`car` returns the first element of a list and `cdr` returns the rest. Both answer `nil` when handed the empty list. That's what lets the usual recursion stop cleanly.

A recursive function over a list tests for the empty list first, then does something with `car` and recurses on `cdr`. The base case is `(null xs)`. Use the identity there for whatever you're accumulating. That's `0` for a sum and `nil` for a list you're building.

`mapcar` applies a function to every element of a list and collects the results. Because Common Lisp keeps functions and variables in separate namespaces, you hand it a function with `#'name` for one defined by `defun` or with a `lambda` form for an anonymous one. `#'` is shorthand for `function`. It asks for the function bound to that symbol. Calling a function you're holding in a variable takes `funcall`. A language with one namespace needs neither, and Part 2 builds one.

`defun` defines a function. Its value is the function's symbol. So does `defvar`, with a variable. Definitions hand back the symbol.

Two kinds of variable live in Common Lisp. They behave differently. That difference is the single most important thing in this problem set. An ordinary `let` binds a **lexical variable**, visible only inside the text of the `let`, and a function written inside that text captures it. The aforementioned `defvar` declares a **special variable**, and rebinding one with `let` changes what every function sees for the duration of that `let`, however far away those functions were written. Two of the 16 cases are built to disagree with each other on this. If you predict both the same way, one of them is wrong.

Required Reading

The 16 cases grade those rules. This manual covers those rules. Teaching the language from nothing takes a tutorial longer than the window. You'd still be reading it on Day 11. So read the book below as you work.

Read *Practical Common Lisp* by Peter Seibel. The complete text is online for free. It teaches standard Common Lisp, which every implementation shares. It's written for people who already program:

```
https://gigamonkeys.com/book/
```

Six of its chapters carry this problem set:

What you're working on	Where to read it
How a form is evaluated, and the three kinds of head	4. Syntax and Semantics
`lambda`, `funcall`, `apply`, and `#'`	5. Functions
Lexical variables, special variables, and closures	6. Variables
`when`, `unless`, `cond`, `and`, `or`	7. Macros: Standard Control Constructs
Atoms, cons cells, `nil`, `car`, `cdr`, `mapcar`	12. They Called It LISP for a Reason: List Processing
Association lists, which is what Part 2's environment is	13. Beyond Lists: Other Uses for Cons Cells

Chapters 4, 12, and 6 in that order cover most of what the 16 cases ask. Read chapter 13 the evening before you start Part 2.

Now the vocabulary hazard. It's the same trap that cost people checks in the last problem set. Chapter 7 is called "Macros: Standard Control Constructs," and it teaches `when`, `unless`, `cond`, `and`, and `or` as macros. The standard agrees. It lists the special operators exhaustively in section 3.1.2.1.2.1 and none of those five is on the list.

CLISP still answers `special-operator` for all of them, under the permission quoted earlier. The grader asks CLISP. So:

```
(and 1 nil 3)
```

has `special-operator` in the `operator` column. Read chapter 7 for what `and` does, and answer the field the way this implementation does. One case turns on it. The `macro` answer is still right for two other cases, which classify one way only.

For the exact behavior of one operator, consult the *Common Lisp HyperSpec*, the standard itself, online and searchable:

```
https://www.lispworks.com/documentation/HyperSpec/Front/index.htm
```

Grader Notation

The `value` field is one line of text describing a Lisp value. The grader compares it letter for letter against what CLISP produces, so the notation is part of the answer. Below is the whole notation, which is more than the 16 cases need. No case produces a ratio, a string, a character, or a nested list, but the rules for them are here so you can tell that a missing row isn't the reason your answer looks odd.

Lisp value	Written as
`7`	`7`
`-3`	`-3`
`nil`	`nil`
`t`	`t`
the symbol `empty`	`empty`
the symbol `*rate*`	`*rate*`
`(1 2 3)`	`(1,2,3)`
`(+ 1 2)`	`(+,1,2)`
`((1 2) (3))`	`((1,2),(3))`
the empty list	`nil`
`3/4`	`3/4`
`"ab"`	`"ab"`
`#\a`	`#\a`
a function object	`#<function>`

Four mechanical rules go with the table. Symbols are written in lower case, the grader's own convention. CLISP reads symbols in upper case and prints them back that way, as `EMPTY`, so don't copy the printer's spelling. Elements of a list are separated by commas and nothing else. No field contains a space anywhere, so `(1, 2)` fails a check that `(1,2)` passes, and a cell padded to line up your columns in an editor fails for the same reason. A function object is always `#<function>`, because CLISP prints a memory address in its own notation and no two runs would agree.

The other three fields are shorter. `kind` is one of the eight words in the table earlier in this manual. `length` is a whole number or the word `none`. `operator` is `function`, `special-operator`, `macro`, or `none`.

If you prefer the rule to the table, `scripts/support.lisp` is the code that writes these descriptions. It's short. The grader and `./run` both call it, so nothing else decides what a description looks like.

Predict, Then Commit

Replace every `TODO` in `predictions.tsv`. Keep the tab-separated layout and the case order. Then check the table's form:

```
./lint
```

`./lint` reads the form only. It never opens `tests/expected.tsv` and never evaluates a case, so it tells you nothing about whether an answer is right, and that's what makes it safe to run before the prediction commit. What it catches is the class of mistake that costs marks for no reason. A padded cell, a space inside a value, a tab your editor replaced with spaces, a kind that isn't one of the eight, a missing or reordered row. Those all survive the parse and then fail their comparisons, so they read like wrong predictions. One trailing space on every line fails all 16 `operator` checks. Each failure prints as `FAIL P01.operator`, a label that never mentions a space.

It reports each fault with its line and column:

```
line 3, column kind: "list" is not one of the course kinds. They are null,
cons, symbol, integer, ratio, string, character, function.
```

Line 3 is P02. The header is line 1.

When it says the table is well formed and complete, write the reasoning entries in `REASONING.md` and commit both files before running any case:

```
git add predictions.tsv REASONING.md
git commit -m "docs: predict CLISP values and operator classes"
```

That commit records your prediction before you inspect a case or open `tests/expected.tsv`. After it exists, you may run:

```
./run P01
```

The runner prints one description:

```
case: P01
value: 7
kind: integer
length: none
operator: function
```

Compare the observation with your row. When they differ, keep the prediction commit and correct the table in a later commit, with the correction entry in `REASONING.md`. Say which rule you got wrong in that commit message:

```
git add predictions.tsv REASONING.md
git commit -m "fix: quote does not evaluate its argument"
```

That history is more useful than a table made perfect after the answers were already on screen.

Part 2: Build the Evaluator

Now you build the thing that produces the behavior you just predicted.

`src/evaluator.lisp` implements a small language written in prefix form, the same way Lisp is. It's much smaller than Lisp. Here is the whole grammar:

```
expr := number
      | symbol
      | (+ expr expr) | (- expr expr) | (* expr expr)
      | (let ((name expr) ...) body)
      | (lambda (name ...) body)
      | (operator-expr argument-expr ...)
```

An **environment** is an association list, a list of `(name . value)` pairs. The empty environment is `nil`. A `lookup` walks the list and takes the first pair that matches. An inner binding therefore shadows an outer one for free, with no extra code.

The toy language differs from its host in one way. It has a single namespace, which makes it a **Lisp-1**. A variable holding a function is callable in operator position, so `(f 1)` works with no `funcall` anywhere. Common Lisp has two namespaces and is a **Lisp-2**. That's why you keep writing `funcall` and `#'` in the code that implements it. `ANALYSIS.md` asks you about that.

The grader scores four groups independently. A pair that reaches E3 and never gets E4 working keeps every mark from E1 through E3. Work in order.

E1: Numbers and Symbols

The two functions below are the smallest evaluator that does anything at all.

`lookup` takes a symbol and an environment and returns the bound value. The first match wins. An unbound symbol is an error. `nil` is itself a legitimate value, so a `lookup` that returns it on failure reads as success. The grader checks both ends of that rule. A binding whose value is `nil` comes back as `nil`. An unbound symbol signals an error.

`eval-expr` takes an expression and an environment. A number evaluates to itself. A symbol goes through `lookup`.

Order the branches with care. `nil` is a symbol as well as the empty list, and a number isn't a cons, so test the atoms before you test for a list.

```
(eval-expr 42 '())              ; 42
(eval-expr 'x '((x . 7)))       ; 7
(lookup 'x '((x . 1) (x . 9)))  ; 1, because the first match wins
```

E2: Prefix Arithmetic

Two more functions bring the recursion.

`eval-operands` takes a list of expressions and one environment, and returns the list of their values. This is `mapcar` over a `lambda`, in one line. Watch the arity when you write it. `eval-expr` takes two arguments and `mapcar` hands its function only one, so the environment comes from the closure.

`apply-op` takes the operator symbol and the list of already-evaluated operands, and does the arithmetic. Anything that isn't `+`, `-`, or `*` is an error.

Then `eval-expr` grows a branch. When the head is one of those three symbols, evaluate the operands and apply. Trace the nesting once:

```
(eval-expr '(* (+ 1 2) (- 10 4)) '())
```

`eval-expr` sees `*`, so it calls `eval-operands` on `((+ 1 2) (- 10 4))`. That maps `eval-expr` over both, which recurses. The first returns 3, the second 6. `eval-operands` hands back `(3 6)` and `apply-op` multiplies to 18. The recursion is the evaluation. That's why the operands finish before the outer operator runs. That's the function rule from Part 1, now written by you.

E3: `let`

The two functions below are where the environment starts changing.

`extend-env` takes a list of symbols, a list of values, and an environment, and returns a new environment with the pairs on the front. `mapcar` takes two lists and `#'cons` pairs them up, so this is also one line. Build a new list. Don't modify the one you were handed. The caller keeps using it after your extension goes out of scope. The grader checks that it's untouched.

`eval-let` handles `(let ((name init) ...) body)`. Evaluate every initializer in the environment you were given, then extend, then evaluate the body in the extended one.

That order is the whole rule. Watch what it buys you:

```
(eval-expr '(let ((x 1))
              (let ((x 2) (y x))
                (+ x y)))
           '())
```

The answer is 3. Both initializers of the inner `let` are evaluated before either symbol is bound, so `y` gets the outer `x` and comes out 1. Evaluate them one at a time, binding as you go. Then `y` would see 2 and you'd get 4. Common Lisp calls the parallel version `let` and the sequential one `let*`. You're building `let`.

The extension also has to end where the body ends:

```
(eval-expr '(let ((x 1)) (+ (let ((x 2)) x) x)) '())   ; 3
```

The inner `let` binds `x` to 2 for its own body. The trailing `x` is still 1. If you get 4 here, the inner bindings outlived the body.

E4: `lambda` ***and Application***

The last two functions carry the idea the whole unit has been building toward.

`make-closure` takes the parameters, the body, and an environment, and returns the list `(:closure params body env)`. That's all a closure is: the code, plus the defining environment.

The environment you pass is the one `eval-expr` holds when it meets the `lambda`. Later call sites are irrelevant. That argument is where lexical scope lives.

`apply-closure` takes a closure and a list of arguments. It pulls the saved environment out of the closure, extends it with the parameters bound to the arguments, and evaluates the body there. It ignores the caller's environment entirely. Signal an error when the argument count doesn't match, and when what you were handed isn't a closure at all.

Then `eval-expr` gets its last two branches. A `lambda` form builds a closure. Anything else with a list head is an application. Evaluate the head to get a closure, evaluate the arguments, and apply. Because the head goes through `eval-expr` like everything else, a variable holding a closure works in operator position, and so does a `lambda` form written inline.

```
(eval-expr '((lambda (n) (* n 2)) 21) '())                    ; 42
(eval-expr '(let ((f (lambda (n) (+ n 1)))) (f 41)) '())      ; 42
```

Now the case that separates a correct E4 from a plausible one:

```
(eval-expr '(let ((x 2))
              (let ((f (lambda (y) (+ x y))))
```

```
                (let ((x 100))
                  (f 1))))
          '())
```

Lexical scope gives 3. Dynamic scope gives 101. Every other form this section shows returns the same either way, so this one is the clearest place they part. It carries weight in the rubric to match. If you get 101, `apply-closure` extends the caller's environment. It must extend the one inside the closure.

The toy language treats functions as ordinary values, so this works too:

```
(eval-expr '(let ((apply-twice (lambda (g) (g (g 1)))))
              (apply-twice (lambda (n) (+ n 10))))
           '())
```

The answer is 21. A `lambda` goes in as an argument, gets bound to `g`, and is called twice in operator position. Compare what the same idea costs you in the host language, where `g` would need `funcall` and the argument would need `#'`. That comparison is one of the questions in Part 3.

Part 3: Explain the Binding

Replace the placeholder in `ANALYSIS.md` with 300 to 450 words written by both partners. The grader counts every space-separated token in the file, so a heading counts too. It fails while the placeholder text is still there. It fails again when the total lands under 300 or over 450. It enforces both limits. Aim for the middle, so one last round of editing leaves you inside the range.

Trace one form through your own `eval-expr`, then answer these:

- Which line captures the defining environment, and what is in that environment at the moment it's captured?
- What would the scope-discrimination case return if `apply-closure` extended the caller's environment instead, and what would that make the language?
- Why does the host CLISP need `funcall` where the toy language doesn't?

Use the code you wrote as evidence. A general description of closures doesn't answer the prompt. Neither does a paraphrase of this manual.

Running the Checks

Run one case while working on predictions:

```
./run P10
```

Run the whole public grader after every coherent change:

```
./check.sh
```

A complete submission ends with:

```
== result ==
91/91 checks passed
```

`91/91` is the complete automated result. The rubric separately assesses your reasoning, your explanation, your commit history, and your workflow runs.

Check the table's form at any time, which reveals no answers:

```
./lint
```

`./run` exits 0 when it prints the case, 64 when it's called with the wrong number of arguments, and 65 when the case id is unknown. `./lint` exits 0 when the table is well formed and complete, and 1 otherwise. `./check.sh` lists the same form faults before it scores, under a `== predictions.tsv form ==` heading. Read that heading first when it appears. A fault under that heading is a typing mistake.

`check.sh` exits 0 only when all 91 checks pass. An error inside one of your functions becomes a labeled failed check while the script keeps going. The message prints under the failure. Start at the first one.

The four evaluator groups are isolated from each other on purpose. Delete `make-closure`. E4's seven checks fail while E1, E2, and E3 stay at full marks. A missing function fails its own checks and leaves the rest green.

One fault in the table's layout stops the grader before it scores anything. The prediction table has to keep its five columns and all 16 ids in order. When it doesn't, the script says so and quits. So a run that ends without a `== result ==` line points at the layout of `predictions.tsv`. A stray tab, a deleted row, or an editor that turned the tabs into spaces explains it.

The supplied workflow runs the same commands on Ubuntu, macOS, and Windows. Push early enough to see all three jobs finish. A local green run is required. The three workflow jobs must also pass from a clean checkout.

Milestones

Of the twelve days in the window, this schedule uses eleven, which reserves the last day for verification. Start on Day 1.

Day	Date	Work
1	Sat Oct 31	Install CLISP, clone the starter, read all 16 forms together, start the reading
2	Sun Nov 1	Fill the table independently, write the reasoning entries, compare answers, commit both before running anything
3	Mon Nov 2	Run P01 to P08. Correct the rules you had wrong
4	Tue Nov 3	Run P09 to P16. Finish the prediction checks
5	Wed Nov 4	E1. Numbers, symbols, and `lookup`

Day	Date	Work
6	Thu Nov 5	E2. Prefix arithmetic. Free lecture session
7	Fri Nov 6	E3. `let` and `extend-env`
8	Sat Nov 7	E4. `lambda` and application
9	Sun Nov 8	The scope-discrimination case, until it returns 3
10	Mon Nov 9	Write `ANALYSIS.md` together. Free lecture session
11	Tue Nov 10	Run `./check.sh`, push, read all three workflow jobs, submit

Your laboratory work overlaps this window and it's heavier than usual. The Lab 3 defense window closes November 8. The Lab 4 week 2 report window closes the same day. Its week 3 window opens November 9. Lab 5 is released November 8. Days 5 through 9 above are the ones with room to move, so if the laboratory takes a weekend, take it from there and not from the end.

Deliverables

Submit one public GitHub repository for the pair containing:

1. A completed `predictions.tsv`
2. A `REASONING.md` with one entry per case, committed with the table
3. The eight completed functions in `src/evaluator.lisp`
4. A 300 to 450 word `ANALYSIS.md`
5. The unchanged public cases, expectations, runner, grader, `scripts/` helpers, and workflow
6. A green `.github/workflows/test.yml` run on the submitted commit
7. A `README.md` that identifies both members and their GitHub usernames, replacing the starter's own

Both members must commit. Use the syllabus's semantic commit prefixes. Commit history should show the prediction step before execution-based corrections, and show each member's implementation or review work.

After the final push, open the repository's commit history on GitHub and copy the full hash of the commit you want graded. One designated member emails both items included in the body in this form:

```
Repository: https://github.com/<your-account>/cmsc-124-ps3-starter
Commit: <full commit hash copied from GitHub>
```

The submitted hash pins the version I grade. The green workflow run must belong to that commit. A later commit doesn't change the submitted version. CC the rest of the pair. Use this subject line, joining each person's entry with `&`:

```
[CMSC 124 Lec] PS3 Group: LastName1, Initials1 & LastName2, Initials2
```

A trio adds its third member in the same format.

Each member also emails `reflection.txt` individually. Identify the work you did, one rule your prediction missed, one part your partner explained, and what you learned. Use:

```
[CMSC 124 Lec] PS3: LastName, Initials
```

For example: `[CMSC 124 Lec] PS3 Group: Sanchez, SM & Jocsing, RA` and `[CMSC 124 Lec] PS3: Sanchez, SM`.

Common Pitfalls

- Predicting `symbol` for `nil`. The course vocabulary answers `null`, and `nil` is the only value that does.
- Predicting `none` for the length of `nil`. It's the empty list, so it's a sequence, so its length is `0`.
- Predicting `macro` for `and`. CLISP answers `special-operator` for it, and for `or`, `when`, `unless`, `cond`, and `case`.
- Predicting `3` for a quoted arithmetic form. `quote` doesn't evaluate anything, so the value is the list.
- Writing a value with spaces in it. The comparison is exact, so `(1, 2)` is a wrong answer to a case whose result is `(1,2)`. Run `./lint` and it prints the line and column of each fault before a check ever scores it.
- Predicting the same answer for the special-variable case and the closure case. They're built to disagree.
- Returning `nil` from `lookup` when the symbol isn't bound. `nil` is a value a lookup can legitimately return, so a caller can't tell that apart from success. Signal an error.
- Evaluating `let` initializers one at a time, binding as you go. That's `let*`, and one case is built to catch it.
- Extending the caller's environment in `apply-closure`. Every arithmetic case still passes and the scope case returns 101.
- Modifying the environment in `extend-env` instead of building a new one. The bindings then outlive the body that introduced them.
- `clisp -q -norc yourfile.lisp` prints the Win32 error on Windows. The install is fine. Use `./run` and `./check.sh`.
- Running the cases before committing predictions. The Git history would no longer establish which answers were predictions.
- Editing `tests/expected.tsv` until the checks pass. The published copy replaces it during grading.

Academic Honesty

Learn Lisp's rules from the required reading, the HyperSpec, and course material. That study is the assignment working as intended. You may also discuss those rules with your assigned partner. The prediction commit, later correction commits, evaluator, and analysis must reflect your pair's own reasoning.

Large language models may explain a language feature, but submitting generated predictions, code, or analysis as your own work is cheating. Each partner must be able to trace any of the 16 forms and explain any of the eight functions without a tool's help. The public answer table means this part relies on an honor rule. Copying it into `predictions.tsv` before doing the work is still cheating even though the file is available to you.

`REASONING.md` belongs in the prediction commit, the one that carries the table. Git shows when each entry appeared. An entry that arrives with the corrections isn't a prediction.

Important Dates

Event	Date
Problem set released	Friday, October 30, 2026
Pairing sheet published	Friday, October 30, 2026
Submission deadline	Wednesday, November 11, 2026

Both lecture slots inside that window, November 5 and November 9, are yours to work in.

Lecture deadlines are strict. The laboratory's prescribed timeline and booking system don't apply.

Grading Rubric

Criterion	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Predictions and Reasoning (20%)	Every case has a reasoning entry in the prediction commit that names the rule behind each predicted field. Every miss has a correction entry that names the misread rule. All 64 published value, kind, length, and operator checks pass by the final commit.	Every case has a reasoning entry in the prediction commit. A few entries restate the answer without a rule, or a few misses have no correction entry.	Many entries are generic or restate the answer, or many misses have no correction entry. A correct table whose reasoning is absent from the prediction commit earns no more than Fair.	`REASONING.md` is missing or empty, or the predictions and the reasoning first appear after execution with no prediction record.
Evaluator, E1 to E3 (20%)	All 17 checks for `lookup`, `eval-operands`, `apply-op`, `extend-env`, `eval-let`, and the matching `eval-expr` branches pass. `extend-env` builds a new environment and `let` binds in parallel.	At least 14 pass. One rule is off, such as sequential binding or an extension that outlives its body.	At least 9 pass. Arithmetic works but the environment handling is unreliable.	Fewer than 9 pass, or `eval-expr` never evaluates a symbol.
Closures, E4 (20%)	All 7 checks pass, including the scope-discrimination case returning 3. Closures capture the defining environment and application extends that one.	At least 5 pass. Closures work for the direct cases but one of arity checking, the higher-order case, or the closure list is wrong.	The scope case returns 101 while direct application works. Closures are built but capture the wrong environment.	Fewer than 3 pass, or `lambda` is not implemented.
Analysis (20%)	The 300 to 450 word explanation traces a form through the pair's own code, points to the line that captures the environment, states what dynamic scope would return and why, and explains the one-namespace difference using the submitted evaluator.	All three questions are answered, with a thinner trace or a weaker link to the code.	The response describes closures in general but doesn't connect them to the submitted evaluator or to the scope case.	Missing, outside the word range, or unable to explain where the environment is captured.
Collaboration and Reproducibility (20%)	Both members make atomic semantic commits. `./check.sh` reports 91/91 locally and the submitted commit is green on Ubuntu, macOS, and Windows.	Both members contribute and the public checks pass, with one workflow or history weakness.	Contributions are markedly uneven, or the final clean-checkout workflow is red despite a mostly working local submission.	One member did the work, the grader was altered, or the submission can't run through `./check.sh`.